DAA Project — Progr Outputs

Design & Analysis of Algorithms · C Programming · All outputs verified

nguage | Q1 · Dijkstra's Algorithm | Q2 · Rod Cutting DP | ximum Subarray

q1_dijkstra.exe — Matrix-based Demo (5 vertices, source = 0)

Decrease & Conquer

SHORTEST DISTANCES — MATRIX O(V²)

Vertex	Distance	Shortest Path
0	0	0
1	7	0 → 2 → 1
2	3	0 → 2
3	6	0 → 2 → 4 → 3
4	5	0 → 2 → 4

HEAP-BASED DEMO — O((V+E) LOG V) — IDENTICAL RESULTS ✓

Vertex	Distance	Shortest Path
0	0	0
1	7	0 → 2 → 1
2	3	0 → 2
3	6	0 → 2 → 4 → 3
4	5	0 → 2 → 4

q1_dijkstra.exe — Task 3: 6-Vertex Graph Trace (source = 0)

Step-by-Step Evolution

GRAPH EDGES: 0→1(2), 0→2(6), 1→3(3), 1→4(8), 2→4(7), 3→5(9), 4→5(4)

Step	d[0]	d[1]	d[2]	d[3]	d[4]	d[5]	Finalized
0 (init)	0	INF	INF	INF	INF	INF	v0
1	0	2	6	INF	INF	INF	v1
2	0	2	6	5	10	INF	v3
3	0	2	6	5	10	14	v2
4	0	2	6	5	10	14	v4
5	0	2	6	5	10	14	v5

FINAL SHORTEST DISTANCES FROM VERTEX 0

Vertex	Distance	Shortest Path
0	0	0
1	2	0 → 1
2	6	0 → 2
3	5	0 → 1 → 3
4	10	0 → 1 → 4
5	14	0 → 1 → 3 → 5

q1_dijkstra.exe — Task 5: Experimental Comparison + Complexity

Performance

MATRIX VS HEAP — RUNTIME (MICROSECONDS)

V	Matrix $O(V^2)$ (μs)	Heap $O((V+E)\log V)$ (μs)	Graph Type
20	0	0	Dense
20	N/A	0	Sparse
50	0	0	Dense
50	N/A	0	Sparse
80	1000	0	Dense
80	N/A	0	Sparse

Matrix version : $O(V^2)$ time, $O(V^2)$ space

Heap version : $O((V+E) \log V)$ time, $O(V+E)$ space

Dense graph ($E \sim V^2$): Heap = $O(V^2 \log V)$ > Matrix = $O(V^2)$

Sparse graph ($E \sim V$) : Heap = $O(V \log V)$ < Matrix = $O(V^2)$

q2_rod_cutting.exe — CLRS Example (rod length = 10) Dynamic Programming

PRICE TABLE

Length	1	2	3	4	5	6	7	8	9	10
Price p[]	1	5	8	9	10	17	17	20	24	30

DP TABLE — MAX REVENUE R[] AND BEST CUT

Length	Price p[]	Max Revenue r[]	Best Cut
0	0	0	0
1	1	1	1
2	5	5	2
3	8	8	3
4	9	10	2 (cut 2+2)
5	10	13	2 (cut 2+3)
6	17	17	6
7	17	18	1 (cut 1+6)
8	20	22	2 (cut 2+6)
9	24	25	3 (cut 3+6)
10	30	30	10 (no cut)

Max revenue for rod of length 10: 30
Optimal Cut: 10 (sell whole rod)

q2_rod_cutting.exe — Small Example (n=4) + Task 5: Cutting Cost

Dynamic Programming

SMALL EXAMPLE — N=4, PRICE: P[1]=2, P[2]=5, P[3]=7, P[4]=8

Length	Price p[]	Max Revenue r[]	Best Cut
0	0	0	0
1	2	2	1
2	5	5	2
3	7	7	1
4	8	10	2 (cut 2+2)

Max revenue for rod of length 4: 10

Optimal Cuts: 2 + 2 (5 + 5 = 10 > sell whole at 8)

TASK 5 — WITH CUTTING COST C = 2

Length	Price p[]	Max Revenue r[] (c=2)	Best Cut
4	9	9	4 (no cut — cost avoidance)
6	17	17	6
7	17	17	7 (no cut)
10	30	30	10 (no cut)

Max revenue (c=2): 30 | Verification c=0: 30 ✓ (matches original)

Time complexity: $O(n^2)$ — two nested loops over n

Space complexity: O(n) — dp[] and cuts[] arrays

Key recurrence: r(n) = max{ p[i] + r(n-i) } for i in 1..n

With cost c: r_c(n) = max(p[n], max{ p[i] + r_c(n-i) - c })

q3_max_subarray.exe — Examples 1, 2, 3 Divide & Conquer

EXAMPLE 1 — CLRS ARRAY (N=16)

Array: [13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]

Algorithm	Sum	Indices	Elements
Divide & Conquer	43	[7, 10]	18, 20, -7, 12
Brute Force $O(n^2)$	43	[7, 10]	✓ matches
Kadane's O(n)	43	[7, 10]	✓ matches

EXAMPLE 2 — ALL NEGATIVE

Array: [-5, -2, -8, -1, -9]

Result: Sum = -1 at Index [3,3] → returns least-negative element ✓

EXAMPLE 3 — ALL POSITIVE

Array: [3, 5, 1, 2, 4]

Result: Sum = 15 at Indices [0,4] → entire array is max subarray ✓

q3_max_subarray.exe — Verbose Recursion Trace (n=9) Recursion Tree

```
ARRAY: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Call: [0..8]
  Call: [0..4]
    Call: [0..2]
      Call: [0..1]
        Base: A[0]=-2
        Base: A[1]=1
      Combine [0..1]: L=-2  R=1  X=-1  →  RIGHT wins (1)
      Base: A[2]=-3
    Combine [0..2]: L=1  R=-3  X=-2  →  LEFT wins (1)
    Call: [3..4]
      Base: A[3]=4
      Base: A[4]=-1
    Combine [3..4]: L=4  R=-1  X=3   →  LEFT wins (4)
  Combine [0..4]: L=1  R=4  X=2   →  RIGHT wins (4)
  Call: [5..8]
    Call: [5..6]
      Base: A[5]=2
      Base: A[6]=1
    Combine [5..6]: L=2  R=1  X=3   →  CROSS wins (3)
    Call: [7..8]
      Base: A[7]=-5
      Base: A[8]=4
    Combine [7..8]: L=-5  R=4  X=-1  →  RIGHT wins (4)
  Combine [5..8]: L=3  R=4  X=2   →  RIGHT wins (4)
Combine [0..8]: L=4  R=4  X=6   →  CROSS wins (6) ✓
Final Answer: Sum = 6  at Indices [3, 6]  →  elements: 4, -1, 2, 1
```

q3_max_subarray.exe — Task 5: Experimental Comparison + Complexity

Performance

D&C VS BRUTE FORCE — RUNTIME (MICROSECONDS)

n	D&C O(n log n) (μs)	Brute Force $O(n^2)$ (μs)	Speedup
1,000	0	0	—
5,000	0	20,000	≫ faster
20,000	1,000	323,000	323×
100,000	4,000	skipped (too slow)	∞

Recurrence : $T(n) = 2T(n/2) + \theta(n)$

Master Thm : a=2, b=2, f(n)=n → $n^{(\log_2 2)} = n^1 = n$ → Case 2

Result : $T(n) = \theta(n \log n)$

Brute Force: $O(n^2)$ | Kadane DP: O(n) — linear optimal